Expert NMPC for Bench2Drive

Milestones M0 and M1: the new repository, the urban NMPC and the behaviour layer, tested offline · carla-expert-nmpc @ b07cfac · 27 October 2026

Summary

- **M0, done.** A new repository, `~/carla-expert-nmpc`, holding the NMPC core and the SimLingo / Bench2Drive scripts, copied from carla-track-control at 8495ae9 (your uncommitted work there was committed first, so the copy has an exact source). It builds against SimLingo's Python 3.8 environment, in which the leaderboard and both agents will run.
- **M1, done offline.** A new NMPC for urban driving (`core/urban`) and, brought forward from M3, the behaviour layer that decides what it is asked to do. All 11 closed-loop scenarios pass: no contact (nearest 1.08 m), the body on the road, every scenario's own goal met.
- **Speed.** One NMPC solve takes about 10 ms once compiled (JIT, cached on disk, loaded in 0.2 s). A behaviour step is 13–19 ms at p95 with one candidate, and up to 49 ms with three solved in parallel.
- **Not yet done.** Nothing has run in CARLA. The offline plant is a kinematic bicycle with actuator lags, not CARLA's Lincoln MKZ; its parameters are still the race-track Model 3's. M2 (the world model, the MKZ's actuation, the expert on an empty route) is next and has started.

1 The new repository (M0)

The repository was started fresh, copying what the expert needs and leaving the race track behind. `PROVENANCE.md` lists where each part came from.

Here	From carla-track-control	Notes
core/track_control/	track_control/	Unchanged: race-track NMPC, pure pursuit, paths, bicycle model, directives. Its 3 tests pass.
core/urban/	new	The urban NMPC and the behaviour layer (section 2).
sim/plant/, sim/urban/	sim/plant/ (bicycle only)	The offline scenarios are new.
bindings/expertctl.cpp	bindings/trackctl.cpp	Python module: paths, vehicle parameters, the urban NMPC.
expert/	carla_bridge/frames.py, actuation.py	CARLA ↔ library frames; the world model and agents go here.
scripts/	scripts/	CARLA, SimLingo, Bench2Drive; env.sh now uses SimLingo's venv.
external/	symlink	Points to the old repository's: CARLA, SimLingo, checkpoints (about 25 GB) shared.

Changes outside the new repository: the snapshot commit 8495ae9 in carla-track-control (on the working branch, not main), and CasADi 3.7.2, pybind11, imageio-ffmpeg and pytest installed into `external/simlingo_venv`. The JIT needs a C compiler at run time (gcc, already present).

2 The urban NMPC

The race-track NMPC models tyre slip, which is undefined at standstill, and handles two cars as ellipses. Bench2Drive needs stop-and-go traffic, traffic rules and interaction, so the new NMPC uses a kinematic bicycle in the route's path coordinates, with grip as a friction circle on the planned accelerations:

```
state x = [s, e_y, e_psi, v, a, δ] input u = [jerk, δ_rate] (′ = d/dt)
β = atan(b·tan δ / L) b: centre of gravity to rear axle, L: wheelbase
s′ = v·cos(e_psi + β) / (1 − κ·e_y)
e_y′ = v·sin(e_psi + β)
e_psi′ = v·cos β·tan δ / L − κ·s′
v′ = a a′ = jerk δ′ = δ_rate
```

Horizon 5 s: steps of 0.1 s for the first second, 0.2 s after (30 steps), integrated by RK4. It is solved by FATROP (an interior-point method with a Riccati recursion over the stages) and warm-started from the previous plan, shifted by the time since. Every constraint is soft, with one slack per group per step. A plan therefore always exists, the least-violating one when a violation can no longer be avoided.

Group	Keeps	Weight
corridor	the body's front and rear corners inside the drivable band [lo(s), hi(s)], 0.15 m from its edges	10^5
movers	the body's 3 circles outside each mover's clearance circle (radius + growth·t + 0.5 m), 8 slots	10^5
ahead	front ≥ 0.5 m behind a lead's rear; front behind stop gates (with room to stop at the last step)	10^5
behind	rear past pass gates (going before a crossing car)	10^5
grip	$a^2 + a_{lat}^2 \le (\mu g)^2$; v ≤ speed limit	10^5
headway	at the last step: room to stop behind the lead were it to brake	10^2

The cost tracks the speed asked for and the line asked for, with small weights on heading, acceleration, jerk, steering rate and lateral acceleration. The speed asked for is tapered, step by step along the shifted previous plan: to standstill at each stop gate along $\sqrt{(2 \cdot a_{ref} \cdot d)}$, to the lead's speed and to the time gap (d / 1 s), and to $\sqrt{(a_{lat,ref}/|\kappa|)}$ in bends ($a_{ref} = 2$, $a_{lat,ref} = 3$ m/s²). The hard bounds are $a \in [-7, 3]$ m/s², |jerk| ≤ 15 m/s³, the steering angle and rate of the vehicle parameters, and v ≥ −1 m/s.

3 The behaviour layer

A local NMPC cannot choose between waiting for a pedestrian and going before them, or between staying behind a parked car and overtaking it: a plan through the middle of an obstacle has no gradient towards either side, so the solver stays there. The scenarios hit exactly this, so the behaviour layer (planned for M3) was built now. At each step it takes a Scene: the ego, the lanes along the route, the actors with predicted trajectories, and the stop lines. Then:

- **Classify** each actor against the ego lane. A *lead* is in the lane ahead going along it (a cut-in counts from when it enters). A *blocker* is a parked car or object in the lane, kept until passed. A *crosser* is a walker, or a vehicle at an angle, entering the lane, with its time window and the span of route it takes. A *mover* is anything else nearby.
- **Build candidates.** `base` yields to every crosser (a gate keeping the front behind it over its window ± 1 s) and waits behind blockers. `pass:<id>` goes before one crosser, only if the car can clear its span in time. `borrow_left` widens the corridor into the left lane round the blockers, only if that lane may be used and oncoming traffic leaves a gap; once the car is out in the other lane it finishes the overtake.
- **Solve** the candidates in parallel, each on its own NMPC, warm-started from its own last plan.
- **Choose** on what each plan does, not on its NMPC cost (the tapered speeds make waiting look free). The score is speed against the untapered speed asked for, comfort, time out of the ego lane, slack, and +300 for a pass. Plans within the hard constraints win; the last choice's score is ×0.9.
- **Rules.** A stop sign is released after 1 s at rest within 4 m of the line. A light turning red (amber) that needs more than 5 (3) m/s² to stop for is driven through. The end of the route is a stop gate.

4 Offline results

`build/urban_scenarios` (also a ctest) drives each scenario in closed loop: the plant is a kinematic bicycle with a 0.25 s lag on the acceleration and a rate-limited steering lag. The behaviour layer steps every 0.1 s and its plan is played at 20 Hz, as in the leaderboard. Actors follow scripts, and are predicted at constant velocity (to standstill when braking) from when they become visible. The pedestrian appears 1.6 s in, 18 m ahead, from behind the van. Contact is an exact oriented-box test; clearance is the true distance between the bodies.

Scenario	Result	Nearest body [m]	Peak a_{lat} [m/s²]	Step p95 [ms]	Step max [ms]	Candidates chosen
curve	ok	–	3.13	14.6	18	base
lead brake	ok	3.00	0.00	14.5	16	base
cut in	ok	2.59	0.01	18.7	27	base
pedestrian behind van	ok	1.16	2.83	35.0	103	base
red light	ok	–	0.00	14.5	20	base
red light dilemma	ok	–	0.00	13.1	15	base
stop sign	ok	–	0.00	13.6	17	base
parked car overtake	ok	1.08	3.49	43.6	85	base>borrow_left>base
parked car wait	ok	1.10	4.55	46.4	99	base>borrow_left>base
crossing yield	ok	3.70	0.00	15.0	22	base
crossing pass	ok	30.15	0.00	49.3	106	base>pass:1>base

All 11 pass. The candidate sequence shows the behaviour layer at work: the overtakes go base → borrow_left → base, and the crossing car that is far enough off in time is passed (base → pass:1 → base). The step time is highest when three candidates are solved at once, and its worst cases (up to about 105 ms) are the first solves after the scene changes abruptly: a new candidate starting cold, or the pedestrian appearing 18 m ahead.

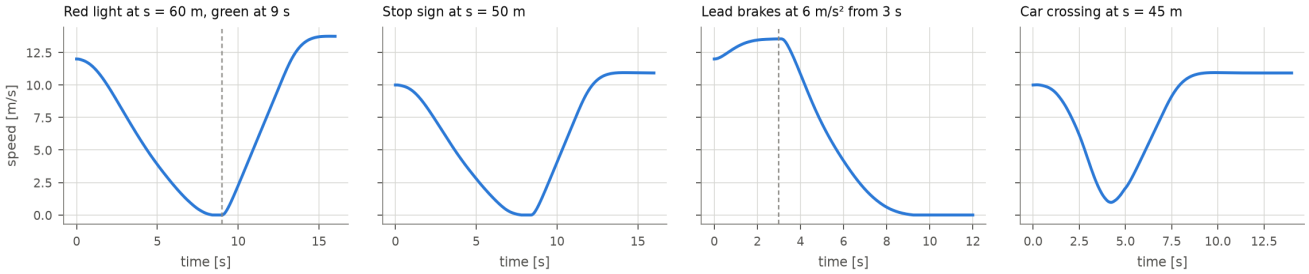


Figure 1. Speed where the car must stop. It drives up and stops, rather than creeping (the speed asked for tapers to the stop). At the light it stops, waits and goes on green. At the stop sign it holds 1 s. Behind the braking lead it stops at the standstill gap. For the crossing car it slows, lets it through, and goes on.

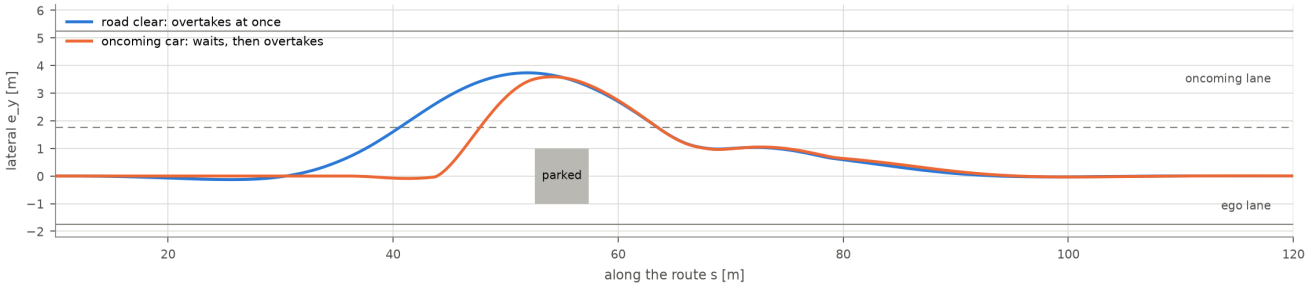


Figure 2. Path round the parked car (grey). With the road clear it moves out and back. With a car coming the other way it waits behind the parked car until the gap is clear, then overtakes. It merges back over about 25 m, not in a jump.

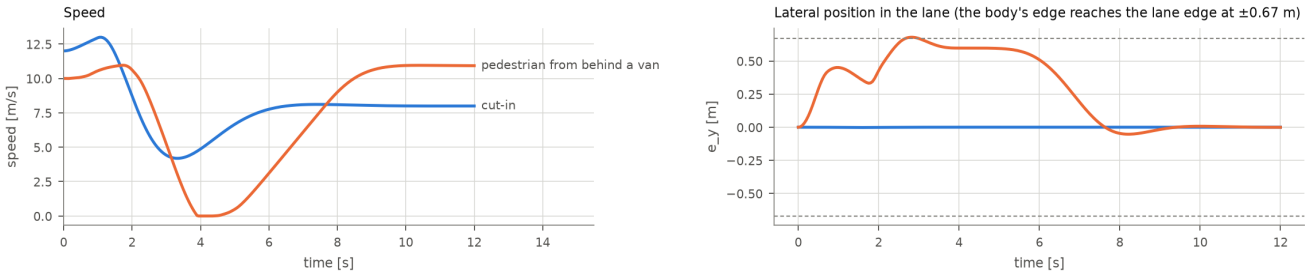


Figure 3. The cut-in and the pedestrian. The car brakes for both and stays in its lane. For the pedestrian it stops, edging to the left side of its lane to keep clear of the van, then goes on once the pedestrian has crossed. Before the time gap was moved into the speed asked for, the cut-in made it swerve out of its lane (section 5).

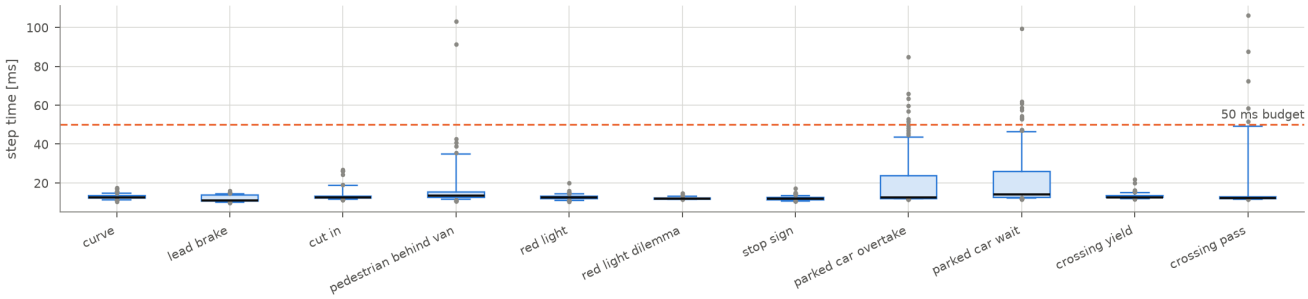


Figure 4. Wall time per behaviour step (box: quartiles; whiskers: 5th–95th percentile; dots beyond). One candidate: 10–20 ms. With two or three candidates in parallel the p95 reaches 40–50 ms, and single steps reach about 105 ms right after an abrupt change (a new candidate starting cold, the pedestrian appearing).

5 What the scenarios taught (design decisions)

The first version failed 7 of 9 scenarios. Each row below is a failure found that way, and the change that fixed it. These are the reasons the formulation looks as it does.

Symptom	Cause	Change
Solves at the 100-iteration limit (≈200 ms) whenever a stop was planned	Dual infeasibility crawls after the plan has converged; degenerate multipliers at standstill	Stop on FATROP's acceptable level (0.1, 3 iterations); $v \geq -1$ instead of $v \geq 0$
Never stops at a red light; creeps towards it	No terminal condition; a convex speed cost spreads a fixed distance over the horizon (Jensen)	Room to stop at the last step; speed asked for tapered to stops, leads and bends
Drives into the pedestrian's path	Slack weight 10^3: a full violation was cheaper than braking	Slack weight 10^5
Plan passes straight through obstacles; stuck behind the parked car	No gradient along the route through a circle's centre (a local minimum)	Behaviour layer: yield/pass gates and lane borrowing as discrete candidates
Overtake never chosen	Waiting costs nearly nothing under tapered speeds	Score candidates on outcomes against the untapered speed
Hits the parked car mid-overtake	Blocker dropped once alongside it; the borrow candidate vanished	Blockers kept until passed; overtake committed once in the other lane
Swerves back at 7 m/s² after an overtake	The corridor jumped back to the lane	The corridor narrows back from where the car is over ≥ 25 m
Swerves during the cut-in	A high-priced position constraint pays the plan to weave (steering shortens progress along s)	Time gap in the speed asked for; only the hard 0.5 m gap as a constraint
Body a few cm past the lane edge	Plans hug the edge; the plant's lag overshoots	0.15 m margin to the corridor edge
27 ms per solve	Function evaluations are 75% of the time	JIT to native code, cached on disk (66 s once, then 0.2 s): about 10 ms

6 Limitations and open points

- **Offline plant.** It is a kinematic bicycle and its parameters are the race-track Model 3's, so these results show the logic works, not that it drives CARLA's MKZ well. M2 identifies the MKZ.
- **Prediction.** The offline actors are predicted perfectly at constant velocity. CARLA's traffic will not be: junction turns, braking, pedestrians stopping.
- **Geometry.** Only straight roads and one bend. No junction geometry, lane changes along the route, right-side nudging, or several blockers in a row. These come with real maps.
- **Step-time margin.** It is thin with three candidates (49 ms at p95), and the first solve after an abrupt change (a new candidate starting cold, an actor appearing close) reaches about 105 ms. The NMPC runs at 10 Hz in a synchronous simulator, so this costs wall time, not safety, but the margin should grow. Options: start a new candidate from the chosen plan rather than cold, fewer mover slots, fewer steps.
- **Circles are conservative laterally.** The ego is covered by 3 circles of radius 1.34 m against a half width of 1.08 m, so tight gaps (a narrow lane past a parked van) are lost.
- **Mode B is not started.** SimLingo's path and speed waypoints have an entry point in the Scene (line_s/line_e, v_ref), but nothing feeds it yet.

7 Next: M2

- **World model** (`expert/world_model`). CARLA to Scene, limited to what a sensor stack could estimate: actors in range and visible (a lidar-point or ray test), velocities from tracked boxes rather than the simulator's, the current state of lights only, lanes from the map. Map-based prediction for vehicles.
- **Actuation for the Lincoln MKZ.** Scripted runs to measure throttle and brake against acceleration over speed, and steer against road-wheel angle. Then an inverse map with a PI loop on the acceleration.
- **The expert agent** (a leaderboard AutonomousAgent). First on an empty route: lane keeping within 0.3 m, smooth stops at lights.

Rebuild this report: `build/urban_scenarios --out results/urban > results/urban/summary.txt; python docs/report/make_m1_report.py`